



Minimization of maximum lateness on parallel machines with sequence-dependent setup times and job release dates

Shih-Wei Lin^a, Zne-Jung Lee^b, Kuo-Ching Ying^{c,*}, Chung-Cheng Lu^d

^a Department of Information Management, Chang Gung University, Taoyuan, Taiwan

^b Department of Information Management, Huaan University, Taipei, Taiwan

^c Department of Industrial Engineering and Management, National Taipei University of Technology, Taipei, Taiwan

^d Graduate Institute of Information and Logistics Management, National Taipei University of Technology, Taipei, Taiwan

ARTICLE INFO

Available online 8 October 2010

Keyword:

Parallel machines scheduling problems

Sequence-dependent setup times

Maximum lateness

Iterated greedy heuristic

ABSTRACT

In this paper, we consider an identical parallel machine scheduling problem with sequence-dependent setup times and job release dates. An improved iterated greedy heuristic with a sinking temperature is presented to minimize the maximum lateness. To verify the developed heuristic, computational experiments are conducted on a well-known benchmark problem data set. The experimental results show that the proposed heuristic outperforms the basic iterated greedy heuristic and the state-of-art algorithms on the same benchmark problem data set. It is believed that this improved approach will also be helpful for other applications.

© 2010 Elsevier Ltd. All rights reserved.

1. Introduction

The scheduling problem addressed in this study aims to minimize maximum lateness on identical parallel machines with sequence-dependent setup times (SDSTs) and job release dates constraints. According to the standard three-field notation introduced by Graham et al. [1], this problem is denoted as $P/ST_{sd}, r_j/L_{max}$. As a result of wide applications in practice, the study of scheduling problems with sequence-dependent setup times or job release dates has attracted considerable attentions in recent years. Surprisingly, while there is a large body of works devoting to the scheduling problems on parallel machines, very few have considered SDSTs or job release dates. Lenstra et al. [2] showed that parallel machine scheduling problems (PMSP) with release dates constraint are *NP-hard in the ordinary sense* whatever the performance criterion considered. Also, Ullman [3] indicated that PMSPs with SDSTs are *NP-hard in the strong sense*. Therefore, the increasing complexity of PMSPs with SDSTs and/or job release dates is the main reason why most of the previous studies did not consider SDSTs or release dates.

Allahverdi et al. [4,5] conducted an extensive review on machine scheduling problems; they revealed that PMSPs with processing characteristics such as SDSTs and release dates remain largely unstudied. Particularly, to our current knowledge, none of the previous studies has attempted to solve the $P/ST_{sd}, r_j/L_{max}$ problem by using exact solution methods. Currently available literature on

this strongly NP-hard problem [6] solely focuses on the development of heuristic (or approximate) solution procedures. One of the pioneering works on the $P/ST_{sd}, r_j/L_{max}$ problem is by Ovacik and Uzsoy [6], who proposed a family of rolling horizon procedures (RHPs) to address this problem. Their computational results showed that RHPs consistently outperformed dispatching rules combined with local search techniques. However, as indicated by Lee et al. [7], some solutions obtained by Ovacik and Uzsoy were infeasible. Separated for many years, Kim and Shin [8] presented a restricted Tabu search (RTS) approach for the same problem. It was found that, on the whole, the performance of the RTS was superior to that of the RHPs, the basic Tabu search (TS), and the simulated annealing (SA). Nevertheless, the performance improvement of RTS is gained at the expense of the extra work on tuning and customization, which may prevent it from being generalized to other problems.

Recently, Lee et al. [7] proposed a restricted simulated annealing (RSA) algorithm that incorporates a restricted search strategy for solving the $P/ST_{sd}, r_j/L_{max}$ problem. Based on extensive computational experiments, Lee et al. demonstrated that the RSA algorithm is highly effective as compared to the best solutions found in the literature on the same benchmark problem data set. The aforementioned state-of-the-art meta-heuristics serve as effective and efficient approaches for solving the $P/ST_{sd}, r_j/L_{max}$ problem. However, these meta-heuristics are very sophisticated algorithm-wise, hence requiring arduous coding efforts for their implementation. For practical purposes, a simple and effective approach is preferable. Therefore, it is necessary to develop approaches that are easy to be comprehended and have a greater potential of being used effectively.

* Corresponding author. Tel.: +866 2 2771 2171; fax: +866 2 2731 7168.
E-mail address: kcying@ntut.edu.tw (K.-C. Ying).

The iterated greedy (IG) heuristic, proposed by Jacobs and Brucos [9], represents one of the simplest meta-heuristics both conceptually and practically. In spite of its simplicity, IG heuristics have been applied successfully to a number of combinatorial optimization problems without relying on too much problem-specific knowledge. For example, satisfactory results have been reported from the applications to set covering problems [9,10] and some scheduling problems [11–16]. With a particular emphasis on developing an algorithm that is easy to be implemented and effective for solving real-world $P/ST_{sd}, r_j/L_{max}$ problems, an improved IG heuristic with a sinking temperature is proposed in this study.

The rest of the article is divided into four sections. The $P/ST_{sd}, r_j/L_{max}$ problem dealt within this study is formulated in Section 2. In Section 3, the proposed IG heuristic is elaborated. In Section 4, the proposed IG heuristic is empirically evaluated by a famous benchmark problem data set, and the comparison of the algorithmic performances against the state-of-the-art meta-heuristics from the relevant literature is reported. Finally, conclusions are drawn together with recommendations for future research in Section 5.

2. Problem formulation

We consider the problem of scheduling n independent jobs $N=\{1, 2, \dots, n\}$ on a set of m identical parallel machines, $M=\{1, 2, \dots, m\}$. Each job $j \in N$ is to be processed by exactly one of the machines, and has a processing time p_j , a release date r_j , and a due date d_j . A sequence-dependent setup time s_{ij} independent of the machine used is required if job j is the immediate successor of job i on a machine. Without loss of generality, we assume the jobs are numbered in non-decreasing order of their release dates and the value of s_{ij} is set to zero when $i=j$. Further, s_{0j} represents the setup time of job j when it is the first job to be processed on a machine.

The objective is to find a feasible schedule $\pi=\{\pi_1, \pi_2, \dots, \pi_m\}$ that minimizes the maximum lateness $L_{max}=\max_{1 \leq j \leq n}\{L_j\}$, where π_k ($k \in M$) is the sequence of jobs on machine k , and $L_j=C_j-d_j$ is the lateness of job j , which is defined as the offset between its completion time C_j and its due date d_j . Moreover, the following assumptions are made for the $P/ST_{sd}, r_j/L_{max}$ problem considered in this study:

- The number of machines is smaller than the number of jobs, i.e., $m < n$ in order to avoid trivial cases.
- All the parameters are deterministic non-negative integers except that the due date of each job is allowed to be a negative quantity.

- All machines are continuously available and each machine can handle one job at a time without preemption.
- Each job can be processed only once on any one machine.

3. Development of the proposed iterated greedy heuristic

Stochastic local search (SLS) algorithms have long been regarded as powerful approaches for solving computationally difficult problems. Some famous SLS such as Tabu search (TS), simulated annealing (SA), memetic algorithms (MA), ant colony optimization (ACO), evolutionary computation (EC), variable neighborhood search (VNS), and iterated local search (ILS) have been successfully applied in many areas of operations research. Among the above SLS methods, iterated greedy (IG) heuristic is a novel method that builds a sequence of solutions by iterating greedy construction heuristics through two main steps: *destruction* and *construction*.

As depicted in Fig. 1, a generic IG heuristic usually starts from an initial solution ξ_0 and then generates a sequence of solutions by iterating the greedy constructive heuristic through the *destruction* and *construction* phases [11]. In the *destruction* phase, a partial candidate solution ξ_d is obtained by removing a fixed number of elements from a current candidate solution ξ . Then, in the *construction* phase, a greedy constructive heuristic is utilized to sequentially insert the removed elements into the partial solution (ξ_d) until a full solution ξ_c is re-constructed. Once a new full solution is re-constructed, an acceptance criterion is applied to judge whether the incumbent solution will be replaced by the new solution or not. The process iterates through the *destruction* and *construction* phases until some termination conditions have been met. In addition, another local search heuristic can be applied before the main loop and the acceptance test to improve the initial solution and the re-constructed full solution, respectively.

In the literature [9–16], a simple simulated annealing-like acceptance criterion with a constant temperature was integrated into IG heuristics. However, an IG algorithm using the acceptance criterion with a constant temperature may be stagnated due to insufficient diversification. As an alternative, in this study, we considered an acceptance criterion with a sinking temperature value that is frequently used in simulated annealing algorithms. Based on the framework of the generic IG heuristic, the solution representation and the main steps of the proposed improved IG heuristic with a sinking temperature, named IG_{ST} , are discussed further in the following subsections.

Procedure <i>Iterated_Greedy</i>	
1	begin
2	$\xi_0 := \text{Generate_Initial_Solution}$
3	$\xi := \text{Local_Search}(\xi_0)$ % optional
4	while termination conditions not meet do
5	$\xi_d := \text{Destruction}(\xi)$
6	$\xi_c := \text{Construction}(\xi_d)$
7	$\xi' := \text{Local_Search}(\xi_c)$ % optional
8	$\xi := \text{Acceptance_Criterion}(\xi, \xi')$
9	return ξ_{best}
10	end

Fig. 1. An outline of a generic IG heuristic.

3.1. Initial solution and solution representation

The initial solution is generated by the heuristic shown in Fig. 2. As that is used by Lee et al. [7], in this study, a solution is represented by a string of numbers consisting of a permutation of n jobs and $m-1$ zeros, which divide the n jobs into m segments. For instance, a solution representation as shown in Fig. 3 can be interpreted as follows. There are 21 jobs to be processed on three machines. The operation sequences of jobs on machines 1, 2 and 3 are 3-20-8-16-9-21, 19-2-4-15-12-11-5-17, and 7-13-14-6-1-18-10, respectively.

3.2. Main steps of the proposed IG_{ST} heuristic

Step 1: Generate initial solution

An initial solution π is constructed by the heuristic presented in Fig. 2. Set the obtained initial solution as the incumbent solution (π^*) and the best solution (π_{best}^*).

Step 2: Destruction and construction phases

- (a) Choose α unrepeatable jobs/machines from the n jobs/machines of π^* randomly. Delete them from π^* and add them into π_R^* in the order they are chosen, where π_R^* is a sequence of the α removed jobs/machines.

- (b) Sequentially reinsert the jobs/machines of π_R^* into π_D^* until a full solution is obtained, where π_D^* is the partial sequence of π^* obtained after the removal of α jobs/machines. While constructing, the new best solution (π_{new}^*) obtained by inserting jobs/machines of π_R^* into all possible positions of the current subsequence is recorded.

- (c) IF $\Delta E = L_{\max}(\pi_{new}^*) - L_{\max}(\pi_{best}^*) < 0$, THEN set $\pi_{best}^* := \pi_{new}^*$ and $\pi^* := \pi_{new}^*$; ELSE_IF $r < e^{(-\Delta E/T)}$, THEN set $\pi^* := \pi_{new}^*$, where $r \in [0, 1]$ is a random number; and T is the current temperature which sinks from initial temperature T_0 and decreases from the last temperature whenever running a fixed number of iterations (I_{iter}), according to the formula $T \leftarrow \lambda T$, where $0 < \lambda < 1$.

Step 3: Stopping criteria

Iterate through the *destruction* and *construction* phases of Step 2 until one of the following termination conditions is met.

1. The current temperature (T) is lower than the final temperature (T_F).
2. The number of reductions in temperature is greater than $N_{non-improving}$, which denotes the maximum number of reductions in temperature over which the best objective function value does not improve.

Procedure Initial_solution

- Step 1. Sort all the jobs by non-decreasing order of their due dates;
- Step 2. Set the initial available time of each machine as zero, that is $A_k = 0$
($k = 1, 2, \dots, m$);
- Step 3. Find the machine k with the smallest A_k ; if tie exists, choose the machine with the smallest machine index;
- Step 4. Find the unscheduled job j with the smallest due date and its release time is no greater than A_k ;
- Step 5. If no job can be obtained then
Choose an unscheduled job j' with the smallest release time;
Schedule job j' on machine k and set the start processing time of job j' on machine k as $R_{j'}$;
Else
Schedule job j on machine k and set the start processing time of job j on machine k as A_k ;
End If
- Step 6. Update the available time of machine k and the completion time of job j (or j') by adding the setup time and processing time of job j (or j');
- Step 7. If no unscheduled job exists, terminate the procedure;
Else go to Step 3;
-

Fig. 2. The procedure of the initial solution heuristic.

[3 20 8 16 9 21 0	19 2 4 15 12 11 5 17 0	7 13 14 6 1 18 10]
Jobs sequence in machine 1	Jobs sequence in machine 2	Jobs sequence in machine 3

Fig. 3. An example of solution representation.

The proposed IG_{ST} heuristic takes advantage of the idea of sinking temperature in simulated annealing. In Step 1, an initial solution π is randomly obtained. Steps 2(a) and 2(b) are the *destruction* and *construction* phases, which can be seen as a perturbation mechanism. In Step 2(c), the Boltzmann function that is commonly used in the annealing process of simulated annealing algorithms is applied to enable the proposed IG_{ST} heuristic to escape from a local minimum. This process is achieved by generating a random number $r \in [0,1]$ and replacing the incumbent solution π^* with π_{new}^* if $r < e^{(-\Delta E/T)}$. Meanwhile, if $L_{max}(\pi_{new}^*) < L_{max}(\pi_{best}^*)$, the probability of replacing π^* with π_{new}^* is set as one. Subsequently, the incumbent solution π^* can be iteratively improved by the destruction and construction phases of Step 2 until one of the termination conditions is met. A small numerical example for one iteration of the proposed IG_{ST} heuristic is presented in Fig. 4 for the sake of clearly illustrating the process.

4. Computational results and discussion

4.1. Test problems

To assess the performance of the proposed IG_{ST} on a standardized basis, this study employed the test problems of Ovacik and Uzsoy [6], which were also adopted by many researchers working on the same problem. Ovacik and Uzsoy generated a benchmark problem set for the $P/ST_{sd}, r_j/L_{max}$ problem and presented two benchmark procedures and three rolling horizon procedures (RHPs) to solve it. Table 1 summarizes the benchmark problem set. As shown in Table 1, each test instance was randomly generated and characterized by different values of n ($n=\{25, 50, 75, 100, 125, 150\}$); the number of machines m ($m=\{2, 4, 6\}$); and the release time range parameter Γ ($\Gamma=\{0.2, 0.6, 1.0, 1.4, 1.8\}$). For each combination of these three parameters, 20 instances were randomly generated. Thus, the test bed in this study was composed of 1800 benchmark instances.

It is worth noting that the due date of each job is allowed to be a negative quantity; that is, some jobs may already be tardy when they arrive at the machines. This is often the case when a job is delayed in the prior stages of the manufacturing process [6]. Besides, each job was assigned a release date drawn from a uniform distribution in which the upper limit of the distribution was the product of the expected makespan of the jobs and Γ .

Consequently, the arrivals of jobs were distributed over a longer period of time as Γ increased.

4.2. Results and discussion

The proposed IG_{ST} heuristic was coded in C language and run on a PC with an Intel Pentium 4 (2.4GHz) CPU and 512MB memory. To determine the appropriate values of these parameters for solving the $P/ST_{sd}, r_j/L_{max}$ problem, extensive preliminary experiments were carried out. Since the value of α is a main factor in destruction and construction phase, in Table 2, we listed the average ratio values (Ave. Ratio (%)) of $|(L_{max}^{IG} - L_{max}^{LB} + \varepsilon)/(L_{max}^{LB} + \varepsilon)|$ of the 1800 benchmark instances, where L_{max}^{IG} denotes the L_{max} value found by the proposed IG_{ST} heuristic with different values of α , L_{max}^{LB} is the L_{max} value found by a lower bound (LB) approach presented later in this section, and ε is a very small positive value. It can be seen from this table that there are no significant differences among different values of $\alpha=3, 4, 5$, and 6; however, there do exist a significant difference between $\alpha=2$ and 3 for the Ave. Ratio (%). Considering both solution quality and computational efficiency, the results of the preliminary experiments demonstrated that the appropriate values of these parameters for the test problem set are: $\alpha=3$, $T_0=30$, $I_{iter}=(n+m-1) \times 40$, $\lambda=0.95$, $T_F=1$, and $N_{non-improving}=25$.

Table 1

Summary of Ovacik and Uzsoy's benchmark problem set.

Design parameters	Values used	Total
Number of jobs	25, 50, 75, 100, 125, 150	6
Number of machines	2, 4, 6	3
Range parameter	0.2, 0.6, 1.0, 1.4, 1.8	5
Number of combinations		90
Problems per combination		20
Total number of problems		1800

Table 2

The Ave. Ratios (%) and CPU times corresponding to different values of α .

	$\alpha=2$	$\alpha=3$	$\alpha=4$	$\alpha=5$	$\alpha=6$
Ave. Ratio (%)	0.415	0.401	0.399	0.397	0.400
Ave. CPU time (s)	65.51	101.77	137.45	176.94	215.63

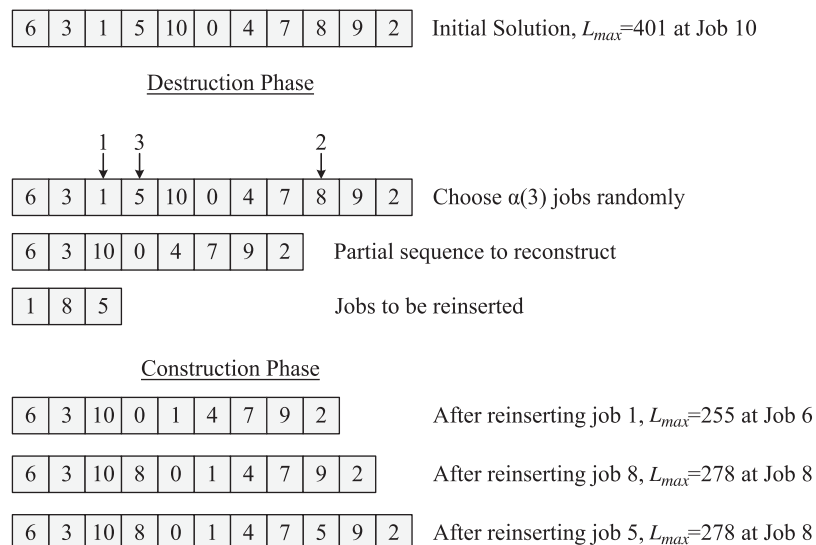


Fig. 4. Example for one iteration of the proposed IG_{ST} heuristic.

which were therefore adopted for all subsequent experiments in this study.

To examine the algorithmic performance, computational results of executing one run of the proposed IG_{ST} heuristic were compared with the best solution (BS_{OU}) and the solutions of the restricted simulated annealing (RSA), the IG heuristic with constant temperature (IG_{CT}), and the corresponding lower bound (LB). The values of BS_{OU} were obtained by Ovacik and Uzsoy [6] using two benchmark procedures and three RHPs, as listed in Table 3. The values of RSA were given by Lee et al. [7]. For the IG_{CT} heuristic, a constant temperature $T = \beta[\sum_{i=1}^n p_i/(n \times 10)]$ was applied in Step 2(c) of the IG_{ST} heuristic, where $\beta(0 < \beta < 1)$ is a parameter that needs to be adjusted. Besides, $N_{non-improving}$ was set as 30 in IG_{CT} and all the experiments were terminated after $(n+m-1) \times (66 \times 40)$ iterations or the best objective function value obtained did not improve after $(n+m-1) \times (30 \times 40)$ iterations. In this study, the best value of β obtained from a preliminary experiment is 0.4.

It is worth noting that the LB approach proposed by Lee et al. [7] for the $P/ST_{sd}, r_j/L_{max}$ problem is wrong. We corrected and simplified the LB approach proposed by Lee et al. [7] as follows:

$$LB = \max_{i=1, \dots, n} \{ \min_{j=1, \dots, l} L_{ij}^{LB} \} \quad (1)$$

where L_{ij}^{LB} denotes the lower bound of job i 's lateness when it is scheduled on the j th ($1 \leq j \leq l$) position and is calculated as follows:

(If $j=1$)

$$L_{ij}^{LB} = \max_{vi} \{ C_i - d_i \} \\ = \max_{vi} \{ (r_i + s_{oi} + p_i) - d_i \} \quad (2)$$

(If $2 \leq j \leq l$)

$$L_{ij}^{LB} = \max_{vi} \{ C_{i,j}^{LB} - d_i \}, \quad \forall j = 2, \dots, l \\ = \max_{vi} \{ [\max(r_i - C_{j-1}^{LB}, R(j) - C_{j-1}^{LB}, 0) + C_{j-1}^{LB} + S_i(0) + p_i] - d_i \}, \forall j = 2, \dots, l \quad (3)$$

In Eq. (3), $C_{i,j}^{LB}$ denotes the lower bound of completion time when a job is scheduled on the position j . Meanwhile, $C_{j-1}^{LB} = R(1) + S(0) + \sum_{k=2}^{j-1} S(k-1) + \sum_{k=1}^{j-1} P(k)$ is the lower bound of completion time when a job is scheduled on the position $j-1$, where $R(k)$ and $P(k)$ ($k=1, \dots, n$) represent the k th smallest value of the release time and processing time, respectively, among all jobs, $S(k)$ ($k=1, \dots, n$) denotes the k th smallest value of the setup time among all s_{ij} ($i=1, \dots, n; j=1, \dots, n; i \neq j$), $S(0)$ is the smallest value of s_{oi} ($i=1, \dots, n$), $S_i(0) = \min_{j=1, \dots, n; i \neq j} s_{ji}$ is the smallest setup time value of job i , and l is the smallest value satisfying the constraint $\max_{i=1, \dots, n} \{s_{oi}\} \leq S(0) + P(1) + \sum_{k=1}^{l-1} [S(k) + P(k+1)]$. The main

Table 3
Algorithms used by Ovacik and Uzsoy [6].

Benchmark procedures	
EDD	EDD dispatching rule
P-EDD-LI	EDD with local improvement
Rolling horizon procedures	
P-EDD-RHP	EDD to assign jobs, RHP to schedule
P- RHP-L	RHP-local
P- RHP	RHP-consider all machines
RHP parameters (κ, λ, T)	
(5, 1, 1)	Low quality/low CPU
(10, 4, 4)	Medium quality/medium CPU
(10, 1, 4)	High quality/high CPU

idea behind this LB approach is that any job scheduled after position l cannot be processed earlier than the job scheduled at the first position if the minimum cumulated setup time and processing time of the first l jobs is greater than or equal to the maximum initial setup time.

The following example demonstrates how the LB for a 10-job and 2-machine problem can be computed.

Example (An Example of LB)

Consider the job data given in Tables 4 and 5. Since

$$\max_{i=1, \dots, n} \{s_{oi}\} = 98 \leq S(0) + P(1) \\ + \sum_{k=1}^{3-1} [S(k) + P(k+1)] \\ = 15 + 23 + [(9+41) + (10+44)],$$

the smallest value of l satisfying the constraint $\max_{i=1, \dots, n} \{s_{oi}\} \leq S(0) + \sum_{k=1}^{l-1} [S(k) + P(k+1)]$ is 3. Therefore, we can calculate $L_{6,j}^{LB}$, $j=1, \dots, 3$, as follows:

$$L_{6,1}^{LB} = C_6 - d_6 = (r_6 + s_{o6} + p_6) - d_6 = (72 + 33 + 54) - (-96) = 255$$

$$L_{6,2}^{LB} = C_{6,2}^{LB} - d_6 \\ = [\max(r_6 - C_1^{LB}, R(2) - C_1^{LB}, 0) + C_1^{LB} + S_6(0) + p_6] - d_6 \\ = [\max\{72 - [18 + 15 + (0) + (23)], 21 - [18 + 15 + (0) + (23)], 0\} \\ + [18 + 15 + (0) + (23)] + 11 + 54] - (-96) \\ = 233$$

$$L_{6,3}^{LB} = C_{6,3}^{LB} - d_6 \\ = [\max(r_6 - C_2^{LB}, R(3) - C_2^{LB}, 0) + C_2^{LB} + S_6(0) + p_6] - d_6 \\ = [\max\{72 - [18 + 15 + (9) + (23 + 41)], 49 - [18 + 15 + (9) \\ + (23 + 41)], 0\} + [18 + 15 + (9) + (23 + 41)] + 11 + 54] - (-96) \\ = 267$$

Therefore $\min_{j=1, \dots, 3} L_{6,j}^{LB} = \min\{255, 233, 267\} = 233$.

Table 4
Job data in the 10-job and 2-machine example.

Jobs	Release time	Processing time	Due date
1	18	23	182
2	21	94	293
3	49	69	122
4	58	48	-80
5	63	44	190
6	72	54	-96
7	89	84	46
8	115	41	237
9	117	68	288
10	134	114	250

Table 5
Sequence-dependent setup times in the 10-job and 2-machine example.

s_{ij}	1	2	3	4	5	6	7	8	9	10
0 ^a	56	83	19	75	41	33	15	98	98	42
1	-	81	59	11	68	11	42	37	81	72
2	10	-	37	34	69	54	55	91	96	29
3	88	94	-	56	12	61	79	100	79	51
4	49	26	58	-	17	72	30	9	32	69
5	55	22	36	26	-	59	62	77	36	25
6	100	54	61	74	82	-	88	69	48	39
7	17	80	41	30	13	93	-	34	88	58
8	97	11	52	78	60	56	43	-	68	81
9	97	23	96	56	38	61	27	70	-	67
10	66	74	37	39	83	61	45	20	46	-

^a Setup time if the job is the first job to be processed in a machine.

A summary of $\min_{j=1,\dots,3} L_{ij}^{LB}, \forall i$ is displayed in Table 6. According to these data, the value of LB can be obtained as follows:

$$LB = \max_{i=1,\dots,10} \{ \min_{j=1,\dots,3} L_{ij}^{LB} \}$$

$$= \max\{-93, -132, 15, 197, -71, 233, 142, -72, -71, 23\}$$

$$= 233$$

As mentioned by Lee et al. [7], some solutions obtained by Ovachik and Uzsoy were infeasible. In order to fairly compare the solutions obtained by the proposed IG_{ST} heuristic with the BS_{OU} , these infeasible solutions were excluded from the data during the following performance comparison. In addition, Kim and Shin [8] proposed a restricted Tabu search (RTS) approach that could efficiently solve the $P/ST_{sd}, r_j/L_{max}$ problem. Nevertheless, as pointed out by Lee et al. [7], some computational results presented by Kim and Shin were wrong. Since the results of Kim and Shin [8] were demonstrated using statistical data, we could not further verify their results or make any comparisons in this study.

The performance of the proposed IG_{ST} heuristic was compared with that of the BS_{OU} , RSA , and IG_{CT} using four indexes: $MAX(\Gamma, n, m)$, $AVE(\Gamma, n, m)$, $BS(\Gamma, n, m)$, and $OPT(\Gamma, n, m)$. $MAX(\Gamma, n, m)$ and $AVE(\Gamma, n, m)$ represent the maximum and average ratio values of $|L_{max}^{Alg} - L_{max}^{LB} + \epsilon| / (L_{max}^{LB} + \epsilon)$ for the problem class characterized by Γ , n and m , where L_{max}^{Alg} denotes the L_{max} value found by a

specific algorithm. $BS(\Gamma, n, m)$ denotes the number of best solutions found by these heuristics for the problem class characterized by Γ , n and m . $OPT(\Gamma, n, m)$ stands for the number of optimal solutions, i.e., the values of L_{max} that reached their LB s, found by these heuristics for the problem class characterized by Γ , n and m . The final results are summarized in Table 7.

As seen in Table 7, the total $MAX(\Gamma, n, m)$, $AVE(\Gamma, n, m)$, $BS(\Gamma, n, m)$, and $OPT(\Gamma, n, m)$ of the proposed IG_{ST} heuristic are 7.884, 0.401, 1322, and 399, respectively. Compared to the corresponding values of 9.719, 0.505, 667, and 101 obtained by the BS_{OU} , the proposed IG_{ST} heuristic outperformed the BS_{OU} on all four of the indexes. Specifically, the improvements are 18.88%, 20.59%, 98.20%, and 295.05% for $MAX(\Gamma, n, m)$, $AVE(\Gamma, n, m)$, $BS(\Gamma, n, m)$, and $OPT(\Gamma, n, m)$, respectively. Similarly, compared to the corresponding values of 8.498, 0.473, 1044, and 342 obtained by the RSA , and the corresponding values of 7.991, 0.404, 1280, and 399 obtained by the IG_{CT} , the proposed IG_{ST} heuristic is superior to RSA and IG_{CT} on all four of the indexes, too.

On the whole, IG_{ST} improves more than half of the benchmark solutions, and 22% of the optimal solutions were found. It can be observed from Table 7 that the number of optimal solutions obtained by the proposed IG_{ST} heuristic increases as Γ increases. For the problem sets of $\Gamma = 0.2, 0.6, 1.0, 1.4$, and 1.8 , the optimal solutions obtained by the proposed IG_{ST} heuristic were 9, 55, 92, 112, and 131, respectively. Comparing these results with those from Table 7, it is evident that the values of LB were getting tighter as Γ increased. Therefore, one may deduce that the actual optimal solutions obtained by the proposed IG_{ST} heuristic were larger than the figures shown in Table 7.

It was also observed from Table 7 that the computational time (CPU time in seconds) of IG_{ST} is slightly larger than those of RSA and IG_{CT} . However, they are so close that this difference can be ignored. Since the computational time required by the BS_{OU} was not provided in the original paper, the current study could not

Table 6
A summary of $\min_{j=1,\dots,3} L_{ij}^{LB}, (i=1, 2, \dots, 10)$.

i	j			$\min_{j=1,\dots,3} L_{ij}^{LB}$
	1	2	3	
1	-85	-93	-43	-93
2	-95	-132	-82	-132
3	15	39	89	15
4	261	197	245	197
5	-42	-71	-28	-71
6	255	233	267	233
7	142	154	171	142
8	17	-72	-72	-72
9	-5	-71	-71	-71
10	40	23	23	23

Table 8
Paired t -tests on ratio of obtained solutions to their LB s.

IG_{ST} v.s.	RSA	IG_{CT}
Difference	-0.073	-0.003
df	1799	1799
t -Value	-16.355	-3.601
One-tail significant	-0.001	-0.001

Table 7
Performance comparison between BS_{OU} , RSA , IG_{CT} , and IG_{ST} .

Problem class	$MAX(\Gamma, n, m)$				$AVE(\Gamma, n, m)$				$BS(\Gamma, n, m)$				$OPT(\Gamma, n, m)$				CPU time (s)		
	BS_{OU}	RSA	IG_{CT}	IG_{ST}	BS_{OU}	RSA	IG_{CT}	IG_{ST}	BS_{OU}	RSA	IG_{CT}	IG_{ST}	BS_{OU}	RSA	IG_{CT}	IG_{ST}	RSA	IG_{CT}	IG_{ST}
(0.2,*,*)	9.719	8.498	7.991	7.884	2.083	1.994	1.717	1.705	111	50	139	170	14	8	8	9	93.29	136.97	151.70
(0.6,*,*)	2.127	2.808	2.140	2.310	0.138	0.252	0.187	0.184	175	163	210	214	16	42	57	55	117.63	112.49	121.91
(1.0,*,*)	0.283	0.464	0.464	0.464	0.060	0.045	0.042	0.041	139	267	300	303	25	79	94	92	89.34	88.60	83.07
(1.4,*,*)	0.310	0.639	0.639	0.639	0.062	0.042	0.041	0.041	130	281	315	316	23	94	111	112	89.46	85.28	74.94
(1.8,*,*)	0.356	0.356	0.356	0.356	0.060	0.035	0.034	0.034	112	291	316	319	23	119	129	131	94.08	85.24	77.23
(*,25,*)	1.159	2.045	2.134	2.096	0.101	0.103	0.106	0.104	170	227	218	224	48	86	85	86	2.29	3.08	2.54
(*,50,*)	2.903	2.540	2.447	2.546	0.174	0.158	0.153	0.148	138	215	221	229	21	78	81	82	12.83	15.07	13.24
(*,75,*)	4.341	3.845	3.812	3.713	0.361	0.322	0.288	0.285	105	182	214	222	10	55	60	60	38.95	43.50	40.21
(*,100,*)	7.623	6.226	5.926	5.911	0.580	0.547	0.459	0.456	91	156	214	216	8	44	55	54	87.36	94.19	91.26
(*,125,*)	8.494	7.108	6.881	6.942	0.779	0.745	0.626	0.622	79	143	213	219	7	40	55	57	167.58	169.12	171.47
(*,150,*)	9.719	8.498	7.991	7.884	0.967	0.967	0.795	0.791	84	129	200	212	7	39	63	60	271.56	285.34	291.89
(*,*,2)	9.719	8.498	7.991	7.884	1.030	0.928	0.852	0.842	218	310	364	410	15	53	59	62	100.39	106.28	103.68
(*,*,4)	3.703	3.941	3.150	3.064	0.333	0.325	0.246	0.247	211	348	466	458	41	134	164	164	96.60	101.65	103.68
(*,*,6)	2.095	2.059	1.467	1.460	0.154	0.167	0.115	0.114	238	394	450	454	45	155	176	173	93.29	97.22	97.95
(*,*,*)	9.719	8.498	7.991	7.884	0.505	0.473	0.404	0.401	667	1044	1280	1322	101	342	399	399	96.76	101.72	101.77

directly compare their computational efficiencies. Nevertheless, even for problems involving up to 150 jobs and six machines, the proposed IG_{ST} heuristic can get very high quality solutions with reasonable computational expenses.

To verify the effectiveness of the proposed IG_{ST} heuristic, the paired t -tests on the ratio of the obtained solutions to their LB s were performed to compare the proposed heuristic with RSA and IG_{CT} . At confidence level $\alpha=0.05$, the results in Table 8 show that the proposed IG_{ST} heuristic significantly outperformed RSA and IG_{CT} . These statistical results clearly show the superiority of the proposed IG_{ST} heuristic. Judging from the experimental results, it is clear that this study has made a step towards reducing the gap between theoretical progress and industrial practice.

5. Conclusions

The $P/ST_{sd}, r_j/L_{max}$ problem is perhaps one of the most notorious NP-hard problems in the strong sense. While retaining both simplicity and generality, this study proposed an effective IG heuristic with a sinking temperature for the $P/ST_{sd}, r_j/L_{max}$ problem. The results obtained clearly reveal that the performance of the proposed IG_{ST} heuristic is superior to that of the highly sophisticated state-of-the-art meta-heuristics and also the basic IG heuristic with constant temperature. The proposed IG_{ST} heuristic combines both the simplicity of the IG heuristic and the cooling mechanism of SA . It is believed that this new approach will also be helpful for other applications, and encourage practitioners to apply it to real-world $P/ST_{sd}, r_j/L_{max}$ problems.

Since $P/ST_{sd}, r_j/L_{max}$ represents a very important but largely unstudied problem, this topic deserves further attentions. The following recommendations are made for future studies. First, it would be worthwhile to apply the proposed IG_{ST} heuristic to other combinatorial optimization problems. Second, extension of the proposed IG_{ST} heuristic to solve this problem with different criteria or multi-criterion may prove both interesting and useful. Third, more studies are necessary to develop additional simple meta-heuristics for this argument. Fourth, it would be worthwhile to develop some exact methods for this problem. Finally, stochastic parallel machine scheduling problems with sequence-dependent setup times and job release dates constraints, while practical, are complex problems requiring further studies.

Acknowledgement

This research was partially supported by the National Science Council of the Republic of China (Taiwan) under Contract nos. NSC 99-2628-E-027-003 and NSC 99-2410-H-182-023-MY2.

References

- [1] Graham RL, Lawler EL, Lenstra JK, Rinnooy Kan AHG. Optimization and approximation in deterministic sequencing and scheduling: a survey. *Annals of Discrete Mathematics* 1979;5:287–326.
- [2] Lenstra JK, Rinnooy Kan AHG, Brucker P. Complexity of machine scheduling problems. *Annals of Discrete Mathematics* 1977;1:343–62.
- [3] Ullman JD. NP-complete scheduling problem. *Journal of Computer and System Sciences* 1975;10:384–93.
- [4] Allahverdi A, Gupta JND, Aldowaisan T. A review of scheduling research involving setup considerations. *Omega: International Journal of Management Science* 1999;27:219–39.
- [5] Allahverdi A, Ng CT, Cheng TCE, Kovalyov MY. A survey of scheduling problems with setup times or costs. *European Journal of Operational Research* 2008;187:985–1032.
- [6] Ovacik IM, Uzsoy R. Rolling horizon procedures for dynamic parallel machine scheduling with sequence-dependent setup times. *International Journal of Production Research* 1995;33:3173–92.
- [7] Lee ZJ, Lin SW, Ying KC. Scheduling jobs on dynamic parallel machines with sequence-dependent setup times. *International Journal of Advanced Manufacturing Technology* 2010;47:773–81.
- [8] Kim CO, Shin HJ. Scheduling jobs on parallel machines: a restricted tabu search approach. *International Journal of Advanced Manufacturing Technology* 2003;22:278–87.
- [9] Jacobs LW, Brusco MJ. A local-search heuristic for large set-covering problems. *Naval Research Logistics Quarterly* 1995;42:1129–40.
- [10] Marchiori E, Steenbeek A. An evolutionary algorithm for large scale set covering problems with application to airline crew scheduling. *Lecture Notes in Computer Science* 2000;1803:367–81.
- [11] Ruiz R, Stützle T. A simple and effective iterated greedy algorithm for the permutation flowshop scheduling problem. *European Journal of Operational Research* 2007;177:2033–49.
- [12] Baraz D, Mosheiov G. A note on a greedy heuristic for flow-shop makespan minimization with no machine idle-time. *European Journal of Operational Research* 2008;184:810–3.
- [13] Ruiz R, Stützle T. An Iterated Greedy heuristic for the sequence dependent setup times flowshop problem with makespan and weighted tardiness objectives. *European Journal of Operational Research* 2008;187:1143–59.
- [14] Ying KC. Solving non-permutation flowshop scheduling problems by an effective iterated greedy heuristic. *International Journal of Advanced Manufacturing Technology* 2008;38:348–54.
- [15] Ying KC. An iterated greedy heuristic for multistage hybrid flowshop scheduling problems with multiprocessor tasks. *Journal of the Operational Research* 2009;60:810–7.
- [16] Ying KC, Lin SW, Huang CY. Sequencing single-machine tardiness problems with sequence dependent setup times using an iterated greedy heuristic. *Expert Systems with Applications* 2009;36:7087–92.